

Live Auction System – Project Documentation

Tech stack: Spring Boot (Java), Redis, Apache Kafka, WebSocket, PostgreSQL, Next.js (TypeScript)

Goal: Build a real-time online auction system where users can place bids and see live updates instantly in the browser.

This document explains the entire project from scratch in simple language, so even a beginner can understand what we built and why.

Table of Contents

1. [What Does This Project Do?](#)

2. [High-Level Architecture](#)

3. [Technologies Used and Why](#)

4. [Project Setup Overview](#)

5. [Step-by-Step Feature Walkthrough](#)

- [Step 1: Basic Spring Boot Project](#)
- [Step 2: Define Auction and Bid Models](#)
- [Step 3: In-Memory Auction Service](#)
- [Step 4: REST API for Auctions and Bids](#)
- [Step 5: Add Redis for Live Auction State](#)
- [Step 6: Atomic Bid Validation in Redis](#)
- [Step 7: Add Kafka Event Publication](#)
- [Step 8: Add WebSocket Broadcasting](#)
- [Step 9: Add CORS Configuration](#)
- [Step 10: PostgreSQL Persistence Consumer](#)

6. [End-to-End Flows](#)

- [Flow 1: Placing a Bid](#)
- [Flow 2: Live Update to Browsers](#)
- [Flow 3: Persisting to PostgreSQL](#)

7. [Running the Project Locally](#)

- [Prerequisites](#)
- [Start Infrastructure with Docker](#)
- [Run Spring Boot Backend](#)

- Run Next.js Frontend
- Verify Everything Works

8. Common Issues and How to Debug

9. How to Extend This Project

10. References

What Does This Project Do?

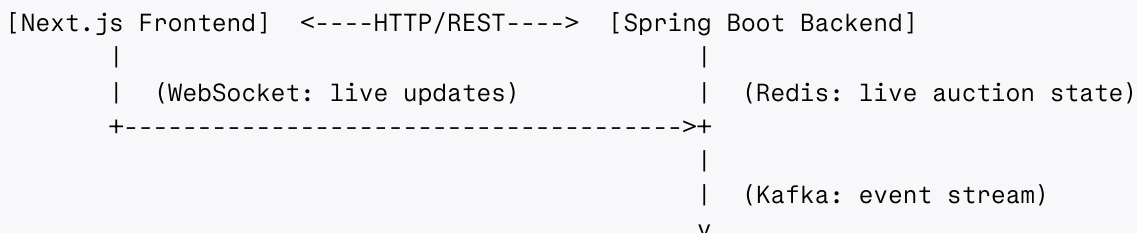
This project is a live online auction system:

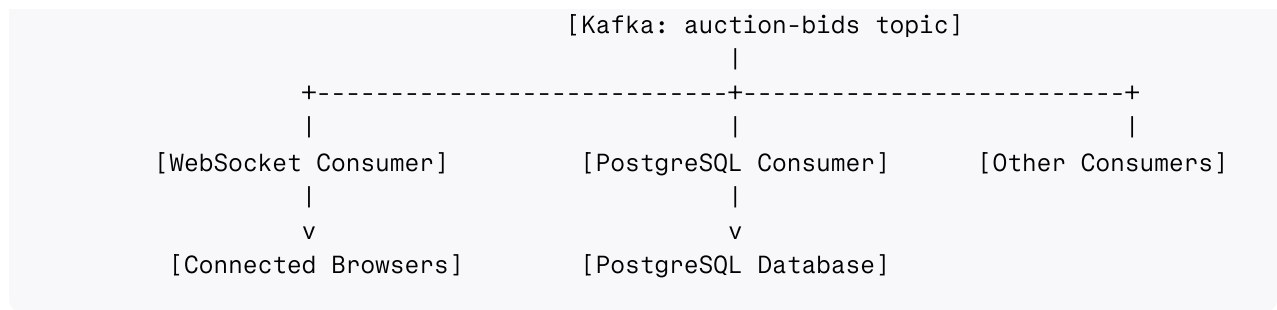
- Sellers can create auctions (items with a starting price and end time).
- Buyers can place bids on active auctions.
- The system:
 - Validates bids (e.g., must be higher than current price, auction must be active).
 - Updates the current highest bid in real time.
 - Broadcasts every accepted bid to all connected browsers instantly.
- Users see:
 - Current price
 - Current highest bidder
 - Bid history
 - Countdown until auction ends
 ...all updating live without refreshing the page.

Behind the scenes, we use:

- **Redis** for fast, atomic updates of auction state.
- **Kafka** as an event bus to decouple components.
- **WebSocket** to push live updates to browsers.
- **PostgreSQL** as the durable database.
- **Spring Boot** for the backend API and event processing.
- **Next.js** for the frontend UI.

High-Level Architecture





Components:

- **Next.js frontend** (localhost:3000)
 - Calls REST APIs to place bids and fetch auctions.
 - Opens a WebSocket connection to receive live bid updates.
- **Spring Boot backend** (localhost:8080)
 - REST controllers for auctions and bids.
 - Uses Redis for atomic, in-memory auction state.
 - Publishes BidAcceptedEvent to Kafka.
 - Consumes from Kafka to:
 - Broadcast to browsers over WebSocket.
 - Persist bids to PostgreSQL.
- **Redis** (in Docker, port 6379)
 - Stores live auction state: current price, highest bidder, status, bid history.
 - Provides atomic operations to avoid race conditions when many users bid at once.
- **Kafka** (in Docker, port 9092)
 - Topic: auction-bids.
 - Acts as the event backbone: once a bid is accepted, an event is published and multiple consumers can react independently.
- **PostgreSQL** (in Docker, port 5432)
 - Durable store for auctions, bids, users, etc.
 - Updated asynchronously by a Kafka consumer.
- **WebSocket layer**
 - Maintains persistent connections to browsers.
 - A Kafka listener reads BidAcceptedEvent and pushes it to all connected sessions for that auction.

Technologies Used and Why

Spring Boot

- Java framework for building REST APIs, event consumers, and WebSocket servers.
- Provides:
 - `spring-boot-starter-web` (REST)
 - `spring-boot-starter-data-redis` (Redis)
 - `spring-kafka` (Kafka)
 - `spring-boot-starter-websocket` (WebSocket)
 - `spring-boot-starter-data-jpa` (PostgreSQL)

Redis

- In-memory key-value store.
- Used for:
 - Storing live auction state.
 - Performing atomic bid validation and update (no race conditions).
- Very fast (microsecond-level reads/writes).

Apache Kafka

- Distributed event streaming platform.
- Used as an **event bus**:
 - After a bid is accepted, we publish a `BidAcceptedEvent`.
 - Multiple consumers react:
 - WebSocket broadcaster.
 - PostgreSQL persister.
 - Future features (notifications, analytics, etc.).
- Decouples components and allows async processing.

WebSocket

- Persistent, bidirectional connection between browser and server.
- Used to **push** live bid updates to browsers instantly.
- Better than polling (no repeated HTTP requests, lower latency).

PostgreSQL

- Relational database.
- Stores:
 - Auctions
 - Bids
 - Users
- Updated asynchronously via Kafka consumers.

Next.js (React + TypeScript)

- Frontend framework.
- Provides:
 - Pages for auction list and auction detail.
 - HTTP client to call backend APIs.
 - WebSocket client to receive live updates.

Docker

- Runs Redis, Kafka, and PostgreSQL locally in containers.
- Ensures:
 - Same environment for everyone.
 - Easy start/stop and cleanup.

Project Setup Overview

Typical local setup:

```
# Docker containers
redis      (port 6379)
kafka      (port 9092)
postgres   (port 5432)

# JVM process
spring-boot-app (port 8080)

# Node process
nextjs-frontend (port 3000)
```

All infrastructure (Redis, Kafka, PostgreSQL) is containerized; your Spring Boot app connects to them via `localhost` ports.

Step-by-Step Feature Walkthrough

Step 1: Basic Spring Boot Project

What we did:

- Created a new Spring Boot project (via Spring Initializr or IDE).
- Added basic dependencies:
 - `spring-boot-starter-web`
- Configured `application.yml` / `application.properties`.

Why:

- Provides the foundation: a runnable Java web application with REST support.

Key files:

- `pom.xml` or `build.gradle` – dependencies.
- `src/main/java/.../AuctionApplication.java` – main class.
- `src/main/resources/application.yml` – configuration.

Step 2: Define Auction and Bid Models

What we did:

- Created Java classes:
 - `Auction` (`id`, `itemId`, `startingPrice`, `currentPrice`, `endTime`, `status`, etc.)
 - `Bid` (`id`, `auctionId`, `bidderId`, `amount`, `timestamp`)
- Possibly added JPA entities for PostgreSQL later.

Why:

- These represent the core domain objects.
- Used throughout the code: controllers, services, Redis, Kafka, DB.

Example:

```
public class Auction {
    private String id;
    private String itemId;
    private BigDecimal startingPrice;
    private BigDecimal currentPrice;
    private Instant endTime;
    private String status; // ACTIVE, ENDED, etc.
    // getters/setters
}

public class Bid {
    private String id;
```

```
private String auctionId;  
private String bidderId;  
private BigDecimal amount;  
private Instant timestamp;  
// getters/setters  
}
```

Step 3: In-Memory Auction Service

What we did:

- Created an AuctionService with:
 - A Map<String, Auction> to store auctions in memory.
 - Methods:
 - createAuction(...)
 - getAuction(id)
 - placeBid(auctionId, amount, bidderId)
- Implemented simple validation:
 - Auction exists.
 - Auction is active.
 - Bid amount > current price.

Why:

- First working version without any external infrastructure.
- Lets us test the core logic before adding Redis/Kafka/WebSocket.

Key class:

```
@Service  
public class AuctionService {  
    private final Map<String, Auction> auctions = new ConcurrentHashMap<>();  
  
    public Auction createAuction(...) { ... }  
    public Auction getAuction(String id) { ... }  
    public Bid placeBid(String auctionId, BigDecimal amount, String bidderId) {  
        // validate and update in-memory state  
    }  
}
```

Step 4: REST API for Auctions and Bids

What we did:

- Created REST controllers:
 - AuctionController
 - GET /api/auctions – list auctions
 - GET /api/auctions/{id} – get one auction
 - BidController
 - POST /api/auctions/{id}/bids – place a bid

Why:

- Exposes our service to the frontend and tools like Postman.
- Standard HTTP/JSON interface.

Example:

```
@RestController
@RequestMapping("/api/auctions")
public class AuctionController {
    private final AuctionService auctionService;

    @GetMapping
    public List<Auction> listAuctions() { ... }

    @GetMapping("/{id}")
    public Auction getAuction(@PathVariable String id) { ... }
}

@RestController
@RequestMapping("/api/auctions")
public class BidController {
    private final AuctionService auctionService;

    @PostMapping("/{id}/bids")
    public ResponseEntity<Bid> placeBid(
        @PathVariable String id,
        @RequestBody PlaceBidRequest request
    ) { ... }
}
```

At this stage, everything is in memory. Restarting the app loses all data.

Step 5: Add Redis for Live Auction State

What we did:

1. Started Redis locally with Docker

```
docker run -p 6379:6379 --name redis -d redis
```

2. Added reactive Redis dependency

In `pom.xml`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

3. Configured Redis in `application.yml`

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
```

4. Stored two sample auctions in Redis

- On startup (e.g., in a `@Component` with `@PostConstruct`), we:
 - Created sample Auction objects.
 - Saved them to Redis using `ReactiveRedisTemplate` or `RedisTemplate`.

5. Changed GET `/api/auctions` to read from Redis

- Instead of the in-memory map, the controller now calls a Redis-backed service.

Why Redis?

- In-memory and very fast.
- Shared state if we run multiple backend instances.
- Natural fit for “live” auction data that changes frequently.

Key concepts:

- **Keys:** e.g., `auction:{id}`
- **Data structures:**
 - Hashes for auction fields.
 - Sorted sets or lists for bid history.

Step 6: Atomic Bid Validation in Redis

What we did:

- Replaced the simple in-memory `placeBid` logic with a Redis-based atomic operation.
- Used either:
 - A **Lua script**, or
 - Redis transactions (`WATCH/MULTI/EXEC`) to ensure validation + update happens as one indivisible operation.

Typical flow in Redis:

1. Read current auction state (price, status, etc.).
2. Check:
 - Is auction active?
 - Is new bid > current price?
3. If valid:
 - Update `currentPrice`, `currentWinner`.
 - Add bid to history.
4. All in one atomic step.

Why atomic?

- Multiple users can bid at nearly the same time.
- Without atomicity, two requests could both see the same old highest bid and both think they win → inconsistent state.
- Redis guarantees that your validation + update run as one indivisible operation.

Example (conceptual Lua script):

```
-- KEYS[1] = auction key
-- ARGV[1] = new bid amount
-- ARGV[2] = bidder id
-- ARGV[3] = timestamp

local auction = redis.call('HGETALL', KEYS[1])
-- parse fields, check status and current price
-- if valid:
--   update current price, winner
--   add to sorted set for history
-- return success/failure
```

In Spring, you execute this via `RedisScript` / `ReactiveScriptExecutor`.

Step 7: Add Kafka Event Publication

What we did:

1. Added Kafka dependency

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

2. Configured Kafka in application.yml

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
```

3. Created a BidAcceptedEvent class

```
public class BidAcceptedEvent {
    private String auctionId;
    private String bidderId;
    private BigDecimal amount;
    private Instant timestamp;
    // constructors, getters/setters
}
```

4. Published event after Redis accepts a bid

In placeBid(...):

```
// after successful Redis update
BidAcceptedEvent event = new BidAcceptedEvent(auctionId, bidderId, amount, Instant.now());
kafkaTemplate.send("auction-bids", event);
```

5. Optionally created the topic automatically

```
@Bean
public NewTopic auctionBidsTopic() {
    return TopicBuilder.name("auction-bids").partitions(3).replicas(1).build();
}
```

Why Kafka?

- Decouples the bid logic from:
 - WebSocket broadcasting.
 - Database persistence.
 - Future features (notifications, analytics, etc.).
- Allows async processing and better scalability.

Now, after a bid is accepted, we simply **announce** “a bid happened” and let others react.

Step 8: Add WebSocket Broadcasting

What we did:

1. Added WebSocket dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

2. Configured WebSocket / STOMP

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic");
        registry.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*"); // CORS handled separately too
    }
}
```

3. Created a Kafka listener that broadcasts to WebSocket

```
@Component
@RequiredArgsConstructor
public class BidEventWebSocketBroadcaster {

    private final SimpMessagingTemplate messagingTemplate;
    private final ObjectMapper objectMapper;

    @KafkaListener(topics = "auction-bids")
    public void handle(BidAcceptedEvent event) {
        // Broadcast to all clients subscribed to /topic/auctions/{id}
        String destination = "/topic/auctions/" + event.getAuctionId();
        messagingTemplate.convertAndSend(destination, event);
    }
}
```

4. Frontend subscribes to WebSocket topics

In Next.js:

```
const socket = new SockJS('/ws');
const stompClient = Stomp.over(socket);

stompClient.connect({}, () => {
```

```

    stompClient.subscribe(`/topic/auctions/${auctionId}`, (message) => {
        const event = JSON.parse(message.body);
        // update UI: current price, highest bidder, etc.
    });
});

```

Why WebSocket?

- Browsers can't subscribe directly to Kafka.
- WebSocket gives a persistent connection so the server can **push** updates instantly.
- Perfect for live auctions, chat, dashboards, etc.

Step 9: Add CORS Configuration

Problem:

- Frontend runs on `http://localhost:3000`.
- Backend runs on `http://localhost:8080`.
- Browsers treat these as **different origins** (because the port is part of the origin).
- Without CORS, the browser blocks JavaScript on `localhost:3000` from reading responses from `localhost:8080`.

What we did:

1. Added CORS config in Spring Boot

```

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}

```

2. Ensured WebSocket endpoint also allows the frontend origin

- Either via `setAllowedOriginPatterns("*")` in WebSocket config, or more specific patterns.

Why?

- So the browser allows:
 - REST calls from Next.js to Spring Boot.
 - WebSocket handshake from Next.js to Spring Boot.

Without this, you'd see CORS errors in the browser console even though everything works in Postman/curl.

Step 10: PostgreSQL Persistence Consumer

What we did:

1. Added JPA + PostgreSQL dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

2. Configured PostgreSQL in application.yml

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/auctiondb
    username: postgres
    password: postgres
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

3. Created JPA entities

- AuctionEntity
- BidEntity

4. Created repositories

```
public interface BidRepository extends JpaRepository<BidEntity, Long> {
    List<BidEntity> findByAuctionId(String auctionId);
}
```

5. Created a Kafka consumer that persists bids

```
@Component
@RequiredArgsConstructor
public class BidPersistenceConsumer {

    private final BidRepository bidRepository;

    @KafkaListener(topics = "auction-bids")
    public void handle(BidAcceptedEvent event) {
        BidEntity entity = new BidEntity();
        entity.setAuctionId(event.getAuctionId());
    }
}
```

```

        entity.setBidderId(event.getBidderId());
        entity.setAmount(event.getAmount());
        entity.setTimestamp(event.getTimestamp());
        bidRepository.save(entity);
    }
}

```

Why async persistence?

- The HTTP request doesn't wait for DB writes.
- Kafka buffers spikes; DB can be slower.
- Keeps the critical bid path fast and resilient.

End-to-End Flows

Flow 1: Placing a Bid

1. Browser → HTTP

- Next.js sends:

```

POST /api/auctions/{auctionId}/bids
Content-Type: application/json

{ "amount": 123.45, "bidderId": "user-123" }

```

2. Controller → Service

- Spring controller calls `placeBid(auctionId, amount, bidderId)`.

3. Service → Redis (atomic)

- Load current auction state from Redis.
- Atomically validate and update:
 - Auction still active?
 - Bid higher than current?
 - Update `currentPrice`, `currentWinner`, `bid history`.
- If invalid → throw exception → HTTP 4xx response.

4. Service → Kafka (publish event)

- On success, publish:

```

BidAcceptedEvent { auctionId, bidderId, amount, timestamp }

```

to topic `auction-bids`.

5. HTTP response

- Return success (and maybe the new current bid) to the browser immediately.

Flow 2: Live Update to Browsers

1. Kafka consumer: WebSocket broadcaster

- Listener:

```
@KafkaListener(topics = "auction-bids")
void handle(BidAcceptedEvent event) {
    messagingTemplate.convertAndSend(
        "/topic/auctions/" + event.getAuctionId(),
        event
    );
}
```

2. WebSocket → Browser

- All browsers subscribed to /topic/auctions/{id} receive:

```
{
  "auctionId": "...",
  "amount": 123.45,
  "bidderId": "user-123",
  "timestamp": "2026-08-29T..."
}
```

3. Frontend updates UI

- Next.js updates:
 - Current price.
 - Highest bidder.
 - Bid history list.
- No page reload; updates appear instantly.

Flow 3: Persisting to PostgreSQL

1. Kafka consumer: persistence

```
@KafkaListener(topics = "auction-bids")
void handle(BidAcceptedEvent event) {
    BidEntity entity = toEntity(event);
    bidRepository.save(entity);
}
```

2. PostgreSQL

- Row inserted into bids table.
- Can be queried later for history, reporting, admin UI, etc.

Running the Project Locally

Prerequisites

- Java 17+ and a JDK installed.
- Maven or Gradle.
- Node.js 18+ and npm/yarn.
- Docker (Docker Desktop or similar).

Start Infrastructure with Docker

Example `docker-compose.yml`:

```
version: '3.8'

services:
  redis:
    image: redis:7
    ports:
      - "6379:6379"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      CLUSTER_ID: "abcdefghijklmnopqrstuv"
    ports:
      - "9092:9092"

  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: auctiondb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
```

Start:

```
docker compose up -d
```

Verify:

```
docker ps
# should show redis, kafka, postgres
```

Run Spring Boot Backend

1. In the backend project directory:

```
mvn spring-boot:run
# or
./mvnw spring-boot:run
```

2. Ensure application.yml points to:

- Redis: localhost:6379
- Kafka: localhost:9092
- PostgreSQL: jdbc:postgresql://localhost:5432/auctiondb

3. Check logs for:

- Redis connection OK.
- Kafka consumer/producer started.
- JPA schema updated.

Run Next.js Frontend

1. In the frontend directory:

```
npm install
npm run dev
```

2. Ensure:

- API base URL points to http://localhost:8080.
- WebSocket URL points to ws://localhost:8080/ws (or similar).

3. Open http://localhost:3000 in your browser.

Verify Everything Works

1. Create an auction (or use the sample auctions loaded on startup).

2. Open the auction detail page in two browser tabs.

3. In one tab, place a bid.

4. Observe:

- Both tabs update instantly with the new price and bidder.
- No page refresh needed.

5. Check PostgreSQL:

- Connect via psql or a DB client.

- Query the bids table; you should see your bid rows.

Common Issues and How to Debug

1. Redis connection refused

- Ensure Redis container is running:

```
docker ps | grep redis
```

- Check port:

```
docker port redis  
# should show 6379 -> 0.0.0.0:6379
```

- Verify application.yml:

```
spring:  
  data:  
    redis:  
      host: localhost  
      port: 6379
```

2. Kafka consumer not receiving events

- Confirm topic exists:
 - Use kafka-topics CLI inside the Kafka container.
- Check logs for:
 - KafkaListener registration.
 - Any serialization errors.
- Ensure BidAcceptedEvent is serializable (e.g., has getters, no complex fields).

3. CORS errors in browser

- Confirm backend CORS config allows http://localhost:3000.
- Check network tab in browser dev tools:
 - Look for OPTIONS preflight request.
 - Inspect response headers for Access-Control-Allow-Origin.

4. WebSocket not connecting

- Check browser console for errors.
- Verify WebSocket endpoint URL matches config:
 - Backend: /ws

- Frontend: `new SockJS('/ws')` or `new WebSocket('ws://localhost:8080/ws')`.
- Ensure CORS allows WebSocket origin.

5. PostgreSQL schema not created

- Check `spring.jpa.hibernate.ddl-auto` setting (update, create, etc.).
- Ensure DB auctiondb exists and credentials are correct.
- Look for SQL errors in logs.

How to Extend This Project

Ideas for next steps:

- **User authentication & authorization**
 - Login/signup.
 - Only authenticated users can bid.
 - Role-based access (admin vs regular user).
- **Auction lifecycle management**
 - Automatically end auctions at `endTime`.
 - Notify winners.
- **Advanced bidding features**
 - Proxy/auto-bidding.
 - Bid increments rules.
 - Reserve prices.
- **Scalability**
 - Run multiple backend instances.
 - Use Redis Cluster / Kafka with multiple brokers.
 - Load balance HTTP and WebSocket connections.
- **Monitoring & observability**
 - Metrics (Prometheus, Micrometer).
 - Distributed tracing (OpenTelemetry).
 - Centralized logging.

References

- Spring Data Redis: <https://docs.spring.io/spring-data/redis/docs/current/reference/html/>
- Spring for Apache Kafka: <https://docs.spring.io/spring-kafka/reference/>
- Spring WebSocket: <https://docs.spring.io/spring-framework/reference/web/websocket.html>

- Redis documentation: <https://redis.io/docs/>
- Apache Kafka documentation: <https://kafka.apache.org/documentation/>
- Next.js documentation: <https://nextjs.org/docs>

End of document.